

Mac から DOFBOT (Raspberry Pi 5) を動かす制御経路 —— 三層の命令, Linux を外した Xinu 版, AIPL 化, シミュレータ, UVC カメラドライバ ——

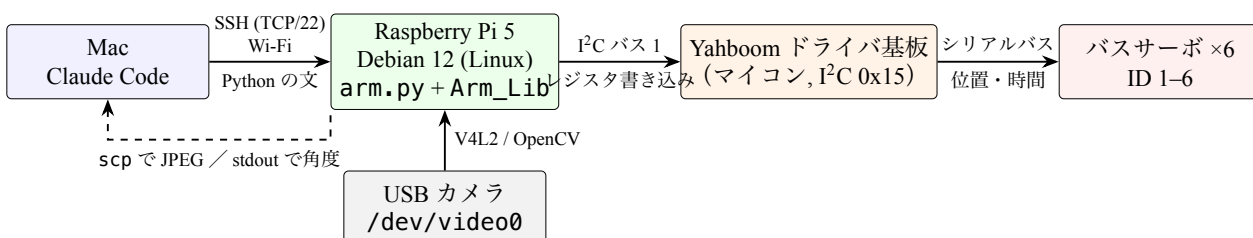
法政大学データサイエンスセンター長 児玉靖司

2026 年 9 月 14 日

要旨

2026 年 9 月 14 日に Mac から実機 DOFBOT を動かした際の制御経路を整理する。命令は「Mac → Linux (Pi 5)」「Linux → ドライバ基板」「ドライバ基板 → サーボ」の三層を経由する。Mac が送るのは Pi 上で実行させる Python コード、Pi が送るのは I²C のレジスタ書き込み、そしてアーム側が理解するのは「サーボ番号・目標角・所要時間」だけである。逆運動学やカメラ処理はすべて上位 (Pi または Mac) の仕事になる。同日午後、Linux を外して同じ Pi 5 に bare-metal の Xinu を載せ、Xinu が RP1 の I²C を直接叩いてアームを動かした (第 6 節)。中間層が Linux + Python から Xinu の C 関数一つに置き換わり、Mac から送る命令は Python コードから URL 一行になった。

1 全体像



2 第 1 層 Mac → Linux (Pi 5) : SSH で Python を送る

Mac 側で実行していたのは、次の形の 1 コマンドである。

```
ssh -o BatchMode=yes pi@192.168.3.18 'cd ~ && python3 -c "  
from arm import *  
move([96,90,15,0,90,30], 800)      # 6 軸の目標角と所要時間 [ms]  
print(angles())                    # 各軸の現在角を読み戻す  
shot(\"/tmp/c96.jpg\")              # カメラで 1 枚撮る  
scp pi@192.168.3.18:/tmp/c96.jpg . # 写真を Mac へ持ち帰る
```

- 経路: Mac → 家庭内 Wi-Fi (TCP/22) → Pi 5 の sshd。Mac の公開鍵を Pi に登録済みなので、パスワードなしで通る。
- 内容: Pi 上の Python を起動し、Pi に置いた /home/pi/arm.py (Yahboom 純正 Arm_Lib の薄い包み+順/逆運動学) を呼ぶ Python の文を文字列として渡す。

- 返答: 標準出力 (角度の読み戻し) と, scp で持ち帰る写真 (JPEG)。

つまり Mac から送っているのは「サーボ角の列」そのものではなく, Pi 上で実行させるプログラムである。arm.py の主な関数を表 1 に示す。

関数	働き
move(pose, ms)	6 軸を同時に pose=[s1..s6] へ ms ミリ秒で動かし, 到達まで待つ
move1(id, deg, ms)	1 軸だけ動かす
angles()	6 軸の現在角を読む (失敗時は 3 回まで再試行)
shot(path)	/dev/video0 から 640×480 を 1 枚撮って保存
ik(x, z, tilt)	把持点 (前方 x m, 高さ z m, 手先傾き tilt°) → (s2,s3,s4)
fk(s2, s3, s4)	サーボ角 → サーボ 5 中心と把持点の (x,z)

Table 1: arm.py の関数

3 第 2 層 Linux (Pi 5) → アーム: I²C でドライバ基板に書く

6 個のサーボは Pi に直接つながっていない。Pi は I²C バス 1 のアドレス 0x15 にある **Yah-boom** のドライバ基板 (マイコン) と会話し, 基板が各サーボとシリアルバスで話す。純正ライブラリ Arm_Lib.py が送っている電文は表 2 のとおりである (Python の smbus で write_i2c_block_data/write_byte_data/read_word_data を呼ぶ)。

目的	レジスタ	データ
サーボ 1 個を動かす	0x10+id	[pos_H, pos_L, time_H, time_L]
6 個を同時に動かす	0x1E → 0x1D	先に [time_H,time_L], 次に [pos_H,pos_L]×6 (12 バイト)
任意 ID (1-250) を動かす	0x19	[id, pos_H, pos_L, time_H, time_L]
角度を読む	0x30+id	0 を書いた後 2 バイト読む (上下バイト入替)
トルク ON/OFF	0x1A	1/0 (0 で手で動かせる)
ブザー	0x06	鳴らす長さ (0.1 秒単位, 0xFF は連続)
RGB LED	0x02	[R, G, B]
基板リセット	0x05	1

Table 2: Pi が I²C (アドレス 0x15) に書く電文

角度から位置値への変換

$$\begin{aligned} \text{サーボ } 1,2,3,4,6 \text{ (0-180}^\circ\text{)} : \quad pos &= 900 + \frac{3100 - 900}{180} \theta \\ \text{サーボ } 5 \text{ (0-270}^\circ\text{)} : \quad pos &= 380 + \frac{3700 - 380}{270} \theta \end{aligned}$$

ただしサーボ 2, 3, 4 は $\theta \rightarrow 180 - \theta$ に反転してから変換する (機構の向きが逆)。time は目標角に到達するまでのミリ秒。

例: 全軸 90° へ 2 秒で

$\theta = 90 \Rightarrow pos = 2000 = 0x07D0$, $time = 2000 = 0x07D0$ なので

```
write_i2c_block_data(0x15, 0x1E, [0x07, 0xD0])
write_i2c_block_data(0x15, 0x1D, [0x07,0xD0, 0x07,0xD0, 0x07,0xD0,
                                     0x07,0xD0, 0x07,0xD0, 0x07,0xD0])
```

(サーボ 5 は 90° が $pos = 1486$ になるので実際は $[0x05, 0xCE]$ 。)

4 第3層 ドライバ基板が受け付ける命令の全体像

Arm_Lib に現れる範囲で、基板は次の種類の命令を受け付ける。

位置指令 サーボ 1 個／6 個同時／任意 ID の「目標角＋所要時間」。速度・トルク値の指定はなく、経路も指定できない（各サーボが独立に目標へ向かう）。

読み出し 各サーボの現在角（0x30+id），サーボの ping（0x38，正常なら 0xDA），ハード版数（0x01）。

設定 トルク ON/OFF（0x1A），サーボ ID の書き換え（0x18），中位オフセットの記録（0x1C／状態 0x1B）。

動作記録（学習モード） ボタンモード（0x03），現在姿勢を 1 コマ記録（0x24），再生モード（0x20: 0 停止／1 一回／2 ループ），記録数（0x22），消去（0x23）。手で動かして覚えさせる用途。

周辺 ブザー（0x06），RGB LED（0x02），製品色（0x04），PWM サーボ別ポート（0x50+id）。

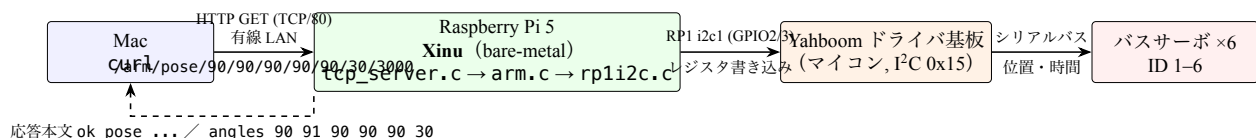
つまりアームが理解するのは「サーボ番号・目標角・時間」だけであり、「掴め」「 $x = 15\text{ cm}$ へ」といった高水準の命令は存在しない。逆運動学・カメラ処理・把持の成否判定はすべて Pi（またはその上流の Mac）側で行う。

5 今回の把持が成功しなかった理由（記録）

- カメラは手首リンク上面にあり、指は小さい映らない。また画像は上下逆に見えた（棚の物が逆さに映り、ベースを左へ回すと像が左へ動く）が、取り付けの向きは実物で未確認。「画像中央に立方体」は「指の間に立方体」を意味しない。
- サーボ 6（グリッパ）の読み戻しは信用できず（閉じた直後に 4 と返ることがある）、把持の成否をセンサで判定できない。
- 目視では指は立方体の手前（アーム側）に降りていた。基板側の問題ではなく、指先位置を観測できないまま角度を推定していたことが原因。

6 Xinu 版：Linux を外して Pi 5 の Xinu が直接動かす

同日、DOFBOT の Pi 5 から Linux の SD を抜き、xinu-rpi5 (USB メモリ XINU5, build Sep 14 2026 08:37:52, commit 9a150bd) で起動した。第 2 層と第 3 層の電文は表 2 のままで、変わったのは「誰が I²C に書くか」である。



第1層 Mac → Xinu: URL 一行

Mac から送るのはプログラムではなく、命令をそのまま並べた URL である（+ や , も区切りとして受け付ける）。

```

B=http://192.168.3.101
curl $B/arm/stat          # I2C の状態
curl $B/arm/ver           # 基板の版
curl $B/arm/ping/3        # サーボ 3 の生存
curl $B/arm/rgb/0/255/0   # LED 緑
curl $B/arm/buzz/3        # ブザー 0.3 秒
curl $B/arm/pose/90/90/90/90/90/30/3000 # 6 軸を 3 秒で
curl $B/arm/set/1/150/1500 # 1 軸だけ
curl $B/arm/read          # 現在角

```

シリアル端末からは同じ命令を `arm pose 90 90 90 90 90 30 3000` のように打てる。HTTP とシェルは同じ `arm_command()` を呼ぶ。

第2層 Xinu → 基板: C 関数が I²C レジスタに書く

Pi 5 のヘッダ GPIO は SoC ではなく RP1 (PCIe 越し) にある。rp1i2c.c は RP1 の i2c1 (0x1F00074000, DesignWare 互換) を GPIO2/3 の FUNCSEL=3 で有効にし、100 kHz のマスタとして全部ポーリングで動かす。割り込みは使わず、すべての待ちに cntpct の期限を置いた。起動時に IC_COMP_TYPE を読み、DesignWare の値 0x44570140 でなければ自分を無効化する (RP1 は無クロックの周辺に 0xDEADDEAD を返す)。

arm.c は Arm_Lib.py と同じ電文を組み立てる。例えば「全軸 90°, グリッパ 30, 3 秒」の pose 命令は、次の 2 回の書き込みになる。

```

/* 時間 3000 ms = 0x0BB8 */
rp1i2c_write(0x15, {0x1E, 0x0B, 0xB8}, 3);
/* ID1..4 = 2000 (0x07D0), ID5 = 1486 (0x05CE), ID6 = 1100 (0x044C) */
rp1i2c_write(0x15, {0x1D, 0x07,0xD0, 0x07,0xD0, 0x07,0xD0,
                    0x07,0xD0, 0x05,0xCE, 0x04,0x4C}, 13);

```

角度の読み出しは「0x30+id に 0 を書く → 3 ms 待つ → repeated START で 2 バイト読む」で、Linux 版と同じ手順を DesignWare の IC_DATA_CMD (CMD_RESTART / CMD_STOP ビット) で組んでいる。

実測 (2026-09-14)

確認	結果
/arm/stat	i2c1 up (IC_COMP_TYPE 一致, FIFO 空・待機)
/arm/ver	board version 0.21
/arm/ping/1~6	すべて 218 (0xDA, ok)
/arm/rgb, /arm/buzz	LED 点灯, ブザー鳴動
起動直後の /arm/read	angles 90 150 ? 0 90 30 (電源断で肩・手首が垂れた姿勢)
/arm/pose/90/90/90/90/90/30/3000	→ angles 90 91 90 90 90 30
/arm/read	
/arm/set/1/30, 150, 90 → /arm/read	30 / 149 / 90
I ² C 統計	転送 106 回, 失敗 0, タイムアウト 0

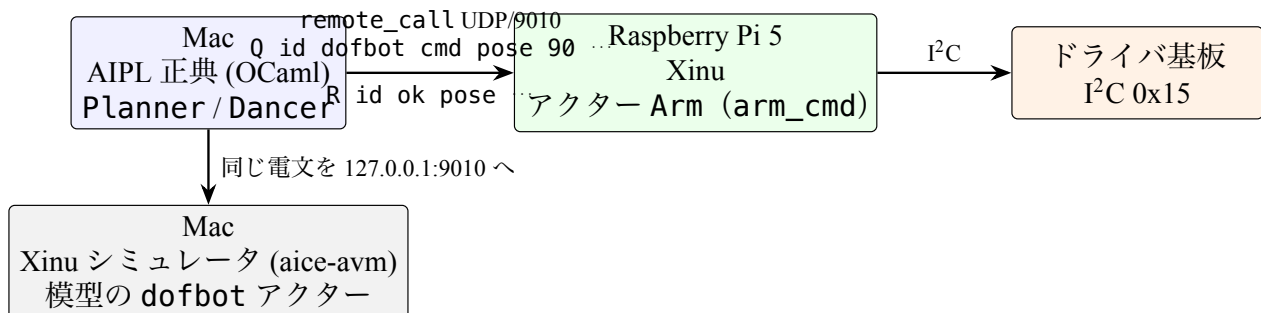
Linux 版との違い

- 中間層が消えた: Linux + Python + smbus が, Xinu の C 関数 2 つ (arm_command → rp1i2c_write) になった。命令から I²C の書き込みまでの経路が全部自分のコードで、待ち時間の上限も自分で決めている。
- Mac が送るものが変わった: Pi で実行させる Python コード → 命令そのものの URL。他の Xinu 板や AIPL の remote から同じ口を叩ける。

- 失ったもの: USB カメラ (Xinu に UVC ドライバがない), Yahboom の ROS2 / AprilTag 認識。観測は当面 Mac か別の Linux 機が担う。
- 注意: 起動後 30 秒ほどは機内ブラウザが airilab.app を取りに行くため HTTP が待たされる (故障ではない)。Linux の SD を挿したままだと SD から Linux が起動する。

7 AIPL 版: 計画は Mac の正典、腕は板の上のアクター

同日午後、命令の発行元を curl から AIPL (OCaml 正典, ~/aios/abclcp) に置き換えた。板の上で走るアクター Arm が腕に触る唯一の存在 (効果 io) で, Mac の計画側は網にしか触らない (効果 net)。



正典に足した組込み

- `remote_call(host:port, actor, method, arg, ms): string` (効果 net) —— 住所を一つ書いて同期に呼び、答を一つ受ける。電文は板の `aipl_remote.c` と同じ ASCII 一行 (`Q <id> <actor> <method> <arg...> /R <id> <値>`), 200 ms ごとに再送, 期限切れは "err". 要求番号は起動時刻からずらす (毎回 20000 から始めると, 相手が「同じ要求の再送」と見て前回の答えを返す。実測)。
- `arm_cmd(s): string` (効果 io) —— 正典では板の `/arm?cmd=` に HTTP で運ぶ。Xinu の機内 AIPL には同名の組込みがあり, `system/arm.c` を直接呼ぶ。
- 実行位置の通知——文を一つ実行するたびに UDP/9011 へ PC 行列アクターファイルを撒く。シミュレータの「AIPL program」窓がその行を反転する。

板の上のアクター (起動時に自動で載る)

```

class Arm {
  var count = 0;
  method cmd(s: string) : string !{io, mut} { count = count + 1; reply(arm_cmd(s)); }
  method served() : int !{} { reply(count); }
}
var arm = new Arm();
web_expose("/dofbot", "arm");

```

再起動でアクターは消える (RAM 上) ので, 起動 45 秒後に板自身が同じ原稿を `abcl2c` → 機内 `cc` で載せ直す。

振り付け (一手=一行)

```

class Planner {
  method dance(d: Dancer, b: string) : int !{net, time, log} {
    var r = 0;
    r = now d.go(b, "pose 90 90 90 90 90 30 1500") timeout 5000 else 0; wait(1750);
    // 直立
    r = now d.go(b, "pose 90 164 18 0 90 30 1500") timeout 5000 else 0; wait(1750);
    // P_LOOK_AT
    ...
    // 全 22 手
    reply(22);
  }
}
class Main {
  method run(p: Planner, d: Dancer) : unit !{net, time, log} {
    var n = now p.dance(d, "127.0.0.1:9010") timeout 120000 else 0; // 1. 模型
    n = now p.dance(d, "192.168.3.101:9010") timeout 120000 else 0; // 2. 実機
  }
}

```

「命令を送る文」と「所要時間だけ待つ文」を同じ行に置く。実行位置の通知は「いま実行中の文」の行を指すので、待っているあいだも腕が動いている手の行が反転し続ける（別行にすると wait の行だけが反転する）。同じプログラムが住所だけ変えて模型にも実機にも向く。実測：模型 22 手 → 実機 22 手, 44 応答すべて ok。

8 Xinu シミュレータの窓

aice-avm のデスクトップ (<http://localhost:8080/>) に二つの窓を足した。

- **DOFBOT arm** : URDF の寸法で描く 3D 模型（サーボ角からの順運動学，ドラッグで回転），6 軸スライダ，模型／実機の切替，読み戻し・同期，右側にカメラ映像。模型は UDP/9010 の dofbot アクターとしても答えるので，AIPL は住所を **127.0.0.1:9010** にするだけで模型が動く。実機への命令は同一生成元の `/api/arm` が板の `/arm` へ中継する。
- **AIPL program** : 走っている `.aipl` を行番号つきで表示し，実行中の文の行を反転（白地）。

9 Xinu の USB カメラドライバ（UVC，等時転送）

カメラ（Microdia 0c45:6340，USB 2.0 高速）を Linux から Xinu の Pi 5 に付け替え，Xinu 自身で映像を受けた。記述子を読むと **YUY2（非圧縮）のみ，MJPEG なし，VideoStreaming は等時（isochronous）エンドポイント 0x81 のみ**で，代替設定 1～6 が 1 μ フレームあたり 128 / 256 / 800 / 800×2 / 800×3 / 1024×3 バイト。もっとも手間のかかる組み合わせである。

流れ

1. 下見（`/usb/cam-probe`）：構成記述子（671 バイト）を読み，VC/VS インタフェース・形式・フレーム・代替設定を表にする。
2. `SET_CONFIGURATION(1)` → VS Probe (`SET_CUR/GET_CUR`, 26 バイト：形式 1, フレーム 3 = 320×240, 間隔 1,000,000×100 ns = 10 fps) → Commit。装置が返す `dwMaxPayloadTransferSize` (1600) 以上の容量を持つ最小の代替設定（alt 4, 800×2）を `SET_INTERFACE` で選ぶ。
3. xHCI に等時 IN エンドポイント（DCI 3, Interval 0 = 125 μ s, MaxBurst 1, Max ESIT payload 1600）を Configure Endpoint で作る。

4. TRB 環 (512 個, 各 3 KB) を満たして doorbell。各 TRB は IOC + ISP + SIA (次に空く μ フレームで送る)。
5. 100 Hz の pump が転送イベントを拾い, UVC ペイロードヘッダ ([0]=ヘッダ長, [1]=FID / EOF / ERR) でフレームを組み立て, EOF で二面バッファの「最新」を差し替える。消費した TRB は同じ位置に積み直す。イベント環は 64 $\rightarrow$ 1024 (10 ms で最大 80 個来る)。
6. 配信: `/cam?w=160&off=N` が YUY2 $\rightarrow$ RGB565 に縮めて 12 KB ずつ返す (CORS つき, /fb と同じ形)。シミュレータの窓が canvas に描く。

実測

確認	結果
<code>/cam/start?frame=3&fps=10</code>	<code>setcfg=1 probe_cc=1 getcur=26 commit_cc=1, alt 4 pkt 1600, cfgep_cc=1 $\rightarrow$ STREAMING</code>
<code>/cam/stat</code> (3 秒後)	frames 29 $\rightarrow$ 76, 完全フレーム 153,600 バイト, overrun 0
起動時自動開始後	frames 393, errs 0
画像	Mac で復元し, 天井 (火災報知器・照明の飾り) が映った。レンズを手で覆うと輝度 173 $\rightarrow$ 96 $\rightarrow$ 171

一度目に失敗した原因

EP0 (制御転送) の転送環 `g_ep0_ring` が全装置で共有されていた。下見が次のコントローラ (起動用 USB メモリ) まで当たった時点でカメラの EP0 の状態が壊れ, 以後の制御転送がすべて時間切れ (`setcfg=0 ...cfgep_cc=19`)。配信開始のたびにカメラをアドレスし直して環を新しくし, 下見は最初の映像装置で打ち切るようにした。

10 その他の実装 (同日)

- ファン (/fan): Pi 5 のファンは RP1 の PWM1 チャンネル 3 (GPIO45)。Linux の `cooling_fan` と同じ周期 41,566 ns・反転極性で, `clk_pwm1` を xosc 50 MHz に設定して回した。10 秒ごとにメールボックス (`GET_TEMPERATURE`) で SoC 温度を読み, Linux と同じ段 (50 / 60 / 67.5 / 75 $\square \rightarrow$ 75 / 125 / 175 / 250) で自動調節する。Xinu では firmware が起動時に決めた状態のまま止まっていた。
- 再起動への備え: 起動 45 秒後に dofbot アクターとカメラ配信を自動で始める。USB の再結線処理は配信中のカメラのポートを触らない。
- 腕基板の固まり: 書きは ACK されるのに読みが全部時間切れになる状態を実測 (22 手中 1 手が I²C 時間切れで抜けたのも同じ兆候)。時間切れは 5 ms 後に一度だけやり直し, `read` が 3 回続けて全軸読めなければ基板をリセット (0x05) する。手動は `/arm/reset`。

11 今後の拡張

腕・カメラ・ファンがすべて Xinu の中で動き, AIPL から一つの口で扱えるようになった。次は, カメラ映像を AIPL の組込み (`cam_frame()`, タグ検出) として計画側に渡し, HDMI 画面にカメラ窓を出し, レポート (9 月 7 日) の設計どおり 6 + 1 アクタに分ける。把持そのものには指の位置を観測する手段 (腕の外から見るカメラか接触センサ) が要る。基板が受け付けるのが角度と時間だけである以上, どの経路でも最終的に流れる電文は表 2 のものになる。

参考: `/home/pi/colcon_ws/src/Arm_Lib/Arm_Lib/Arm_Lib.py` (V1.0.1), `/home/pi/arm.py` (写し `~/dofbot_pi5/arm.py`), URDF: `j3soon/OmniIsaacGymEnvs-DofbotReacher`
`thirdparty/dofbot_info/urdf/dofbot.urdf`, Xinu 版: `xinu-rpi5 device/i2c/rp1i2c.c`,

system/arm.c (9a150bd), UVC: device/usb/rp1usb.c (cddba90), ファン:
device/genet/rp1fan.c (2760a07); 正典: abclcp src/eval_thread.ml (224e9ea); シミュレータ:
aice-avm server.ml, www/js/xinu.js (claac3a); AIPL: ~/dofbot_pi5/aipl/complex.aipl。